

Reproducing an LHC Reinterpretation Pipeline

A From-First-Principles Guide to Running MadGraph $\rightarrow$ Pythia8 $\rightarrow$ Delphes
 $\rightarrow$ SimpleAnalysis $\rightarrow$ pyhf in Containers, and What We Learned Doing It Manually

HEP Agentic Pipeline — pedagogical track
for the project lead / supervisor review

June 8, 2026

Contents

How to read this guide	3
1 Introduction	4
1.1 The goal in one paragraph	4
1.2 Why we ran it manually first	4
1.3 Two documentation tracks (a design principle)	4
2 The pipeline, stage by stage	5
2.1 MadGraph — the hard process	5
2.2 Pythia8 — parton shower, hadronisation, decays	5
2.3 Delphes — fast detector simulation	5
2.4 SimpleAnalysis — the published event selection	6
2.5 pyhf — the statistical limit	6
3 Faithful reproduction: mapyde and containers	7
3.1 Why a single orchestrator	7
3.2 Why we chose the container path	7
4 Infrastructure on Apple Silicon: the procedural journey	8
4.1 Gap 1 — conda's podman is the wrong architecture	8
4.2 Gap 2 — the macOS virtualization helper	8
4.3 Gap 3 — running amd64 images on arm64	8
4.4 Gap 4 — interrupted runs leave stale containers	8
4.5 Gap 5 — the laptop went to sleep	9
4.6 What <i>did</i> work first time	9
5 The trial runs	10
5.1 The 1k smoke test	10
5.2 The 50k production run	10
6 Sensibility and sanity analysis	11
6.1 What a sane result looks like	11
6.2 Comparison to the published reference	11
6.3 Did we reproduce the paper?	11
7 Generalising the pipeline: Rivet, plots, and Contur	12
7.1 Why a plot is the deliverable	12
7.2 Worked demonstration: a gluino vs an ATLAS jets+MET search	12
7.3 The exclusion stage: Contur, and its scope	13
7.4 Two co-equal analysis routines: Rivet and SimpleAnalysis	13

8	The statistical exclusion: pyhf, the data, and the limit	14
8.1	Where the numbers come from	14
8.2	Two model modes	14
8.3	The $\mu = 2$ lesson: a fixed grid is not a limit	15
8.4	Plots a scientist can read	15
8.5	Rigour without a reference	15
9	The agentic trial: dogfooding the workflow	16
9.1	The case, and why	16
9.2	What the agent produced	16
9.3	Where the instructions stumbled (and the fixes)	16
10	Observations & lessons for agentic-workflow design	18
11	Limitations & next steps	19
12	Validation, audit, and readiness	20
12.1	Acceptance validation against published cutflows	20
12.2	An independent cross-check	20
12.3	A self-auditing readiness score	20
A	Glossary	21
B	Reproduce these runs	22
C	References	23

How to read this guide

This is the *pedagogical* document: it explains the physics, the software, and — most importantly — the *procedural reality* of running the full pipeline by hand, including every obstacle hit and how it was solved. It is intended for a human (a project lead or a newcomer with roughly first-year undergraduate physics/programming) who wants to *understand and audit* the work. It is deliberately separate from the *agent track* (`../workflow/`), which contains only terse operational instructions for an executing agent. Nothing here is read by the agent.

A companion document, `../stages/01-event-generation/docs/pedagogical-guide.pdf`, covers the MadGraph (event-generation) stage in much deeper physics detail; this guide focuses on the *whole* pipeline and the *experience* of running it.

Chapter 1

Introduction

1.1 The goal in one paragraph

The Large Hadron Collider has produced many searches for new particles. Each is published against a few benchmark models, leaving most of the theory space untested. *Reinterpretation* re-uses a published search to test a *new* model: simulate that model’s signal, push it through the same analysis the experiment used, and ask whether the data would have excluded it. This guide documents reproducing such a pipeline end-to-end — the one from “*Reduce, Reuse, Reinterpret*” (arXiv:2306.11055), implemented by the `mapyde` package.

1.2 Why we ran it manually first

The project’s aim is an *agentic* workflow: an LLM agent that drives this pipeline from a prompt. Before writing instructions for an agent, we ran the entire pipeline by hand. Doing so surfaces the procedural gaps that a “from reading the paper” description would miss — and those gaps (Chapter 4) are exactly what the agent instructions must handle. This guide is the record of that manual run.

1.3 Two documentation tracks (a design principle)

Everything we produce falls into one of two strictly separated tracks:

- **Agent track** (*workflow/*) — minimal, imperative, general instructions for a (possibly weak) model to *execute*. No physics, no “why”.
- **Pedagogical track** (this document) — exhaustive explanation for a human to *understand*.

Mixing them is the most common failure when extending the project: an agent file bloated with physics loses the thread; a tutorial stripped of “why” teaches nothing. We keep them apart, with no cross-references from agent files into pedagogical ones.

Chapter 2

The pipeline, stage by stage

The pipeline is a chain of five tools, each consuming the previous one’s output file (Figure 2.1). Below, what each does and why.

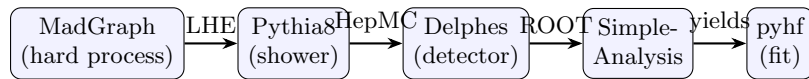


Figure 2.1: The five-stage pipeline and the file passed between stages.

2.1 MadGraph — the hard process

MadGraph computes the fundamental interaction: it draws the Feynman diagrams for the requested process (here slepton-pair production), generates code for the squared matrix element, integrates it over phase space to get a cross-section, and emits unweighted parton-level events (an LHE file). In `mapyde`’s container, Pythia8 is bundled with MadGraph and runs immediately after, so this stage already produces a showered HepMC file. Deeper detail: the companion stage-1 guide.

2.2 Pythia8 — parton shower, hadronisation, decays

The bare quarks/gluons and sleptons from MadGraph are not what a detector sees. Pythia8 evolves them: it radiates additional gluons/quarks (the *parton shower*), binds quarks into hadrons (*hadronisation*), and decays unstable particles (including the sleptons $\rightarrow$ lepton + neutralino). It also performs the *CKKW-L merging* that stitches together the different jet-multiplicity matrix elements without double counting. Output: a HepMC event record of particle-level final states.

2.3 Delphes — fast detector simulation

A real detector measures particles with finite efficiency and resolution. Delphes applies a parametrised model of the ATLAS detector (tracking, calorimetry, lepton identification efficiencies) to the particle-level events, producing reconstructed physics objects (electrons, muons, jets, missing transverse momentum) in a ROOT file. The slepton analysis is especially sensitive to the *low-momentum lepton efficiencies*, which the ATLAS Delphes card encodes.

2.4 SimpleAnalysis — the published event selection

`SimpleAnalysis` is ATLAS’s own tool that applies a published analysis’s exact selection (cuts on lepton p_T , missing momentum, invariant mass $m_{\ell\ell}$, the stransverse mass m_{T2} , ...) and counts how many signal events land in each *signal region*. We use the `EwkCompressed2018` selection (the compressed-SUSY search the paper reinterprets). It runs in an ATLAS-provided container; reproducing it faithfully is the hardest part of the chain to set up (Chapter 4).

2.5 pyhf — the statistical limit

Given the signal yields per region and the experiment’s published probability model (a `pyhf` JSON “workspace” with the backgrounds, data, and all systematic uncertainties), `pyhf` performs the hypothesis test and returns the 95% confidence-level upper limit on the *signal strength* μ — the factor by which the model’s cross-section would have to be scaled to be excluded. $\mu < 1$ means the nominal model is excluded; $\mu > 1$ means it is not.

Chapter 3

Faithful reproduction: mapyde and containers

3.1 Why a single orchestrator

Each stage has its own software, versions, and data cards. `mapyde` ties them together: one TOML configuration file describes the whole job, and `mapyde` runs each stage in a pinned *container* (a packaged, reproducible software image). This is why the paper’s results are reproducible at all — the exact MadGraph 2.9.3, Delphes, ATLAS SimpleAnalysis, and pyhf images are fetched and run identically on any machine with a container runtime.

3.2 Why we chose the container path

We deliberately chose the faithful containerised path (`mapyde` + the real ATLAS containers) over re-implementing stages natively. It is the most honest reproduction, and — crucially for the project — discovering whether it works on a fresh machine is itself a finding. It did work, but only after clearing several non-obvious obstacles, which is the subject of the next chapter.

Chapter 4

Infrastructure on Apple Silicon: the procedural journey

This is the heart of “what we learned manually.” The host was a macOS Apple-Silicon (arm64) laptop with *no* container runtime and *no* administrator rights. Getting the paper’s amd64 containers to run there took five distinct fixes. Each is a procedural gap an agent must handle, and each is now encoded in `workflow/checklists/troubleshooting.md`.

4.1 Gap 1 — conda’s podman is the wrong architecture

The obvious move, `conda install podman`, gives an *x86_64* binary. Run under Rosetta it reports *amd64*, so `podman machine init` downloaded an amd64 virtual-machine image. Apple’s Virtualization framework can only boot a *native* (arm64) guest, so `podman machine start` failed with `VZErrorDomain Code=1 "Internal Virtualization error"`. **Fix:** install the *native arm64* podman client from the official GitHub release zip. Then `init` pulls an arm64 image and the VM boots.

4.2 Gap 2 — the macOS virtualization helper

`podman machine` on macOS needs two helper binaries: `gvproxy` (networking, available from conda-forge) and `vfkit` (the bridge to Apple’s Virtualization framework, *not* on conda-forge).

Fix: download the signed `vfkit` release binary (it carries the required `com.apple.security.virtualization` entitlement and runs without admin) and point podman at it via `CONTAINERS_HELPER_BINARY_DIR`.

4.3 Gap 3 — running amd64 images on arm64

The pipeline images are built for `linux/amd64` only. A plain `podman run` on arm64 looks for an arm64 variant and fails. **Fix:** run them under emulation. The podman machine’s Linux VM already has the emulation layer, so `--platform linux/amd64` works; we added a thin `podman` wrapper that injects that flag so `mapyde`’s plain `podman run` calls `emulate` transparently. The cost is speed: emulated containers are several times slower than native.

4.4 Gap 4 — interrupted runs leave stale containers

`mapyde` names each stage’s container with a fixed string (e.g. `output_mgpy`). If a run is interrupted, that container persists and the next run fails with “name already in use.” **Fix:** remove stale containers before each run (our driver does this automatically).

4.5 Gap 5 — the laptop went to sleep

A long emulated run was killed when the Mac idle-slept during an unattended gap, taking the VM (and the in-flight stage) with it. **Fix:** wrap long runs in `caffeinate -i -s`, which holds `PreventUserIdleSystemSleep/PreventSystemSleep` for the run’s duration.

4.6 What *did* work first time

Not everything was a fight. All four images — including the ATLAS SimpleAnalysis container from CERN’s GitLab registry — were publicly pullable with no authentication. `mappyde` bundles the process/Pythia/Delphes cards *and* the slepton `pyhf` likelihood, so no external data download was needed. And once the VM booted, amd64 emulation worked out of the box.

Lesson for the agent workflow. The five gaps above are invisible if you only read the paper; they only appear when you actually run it on a constrained machine. This is exactly why we ran it manually before writing the agent instructions.

Chapter 5

The trial runs

We ran two trials at the slepton point $m(\tilde{\ell}) = 200$, $m(\tilde{\chi}_1^0) = 150$ GeV ($\Delta m = 50$ GeV): a 1,000-event *smoke test* and a 50,000-event *production* run. Both used the identical faithful configuration (MadGraph 2.9.3, EwkCompressed2018, the bundled Slepton likelihood); only the event count differed.

5.1 The 1k smoke test

Purpose: validate that all six stages run end-to-end. **Result:** they did (madgraph ≈ 24 min, delphes 61 s, Delphes2SA 10 s, SimpleAnalysis 8 s, sa2json 22 s, pyhf ≈ 19 min; ~ 45 min total under emulation). MadGraph’s LO cross-section for the 1-jet ISR slepton sample was **20.6 fb**; SimpleAnalysis total acceptance **2.4%**. But with only ~ 24 events passing selection, scattered one-per-signal-region, the statistics were far too thin to set a limit: the μ scan hit its ceiling ($\mu \geq 2$, $\text{CL}_s = 0.38$). *This is the expected outcome of a smoke test* — it proves the plumbing, not the physics.

5.2 The 50k production run

Purpose: obtain statistically meaningful yields and a real limit. **Per-stage timing (amd64 under emulation):** madgraph+Pythia ≈ 7.9 h (the dominant cost), Delphes ≈ 42 min, Delphes2SA 42 s, SimpleAnalysis 19 s, sa2json 20 s, pyhf 18 min; **~ 8.9 h total**. **MadGraph cross-section (LO, 1-jet isrslep):** 20.47 fb — stable against the 1k run’s 20.55 fb, confirming the generation is reproducible. **SimpleAnalysis:** baseline acceptance 2.42%, but each finely-binned signal region receives $\leq 8 \times 10^{-5}$ acceptance. **Signal reaching pyhf:** only **7.0 expected events** across 32 signal regions (≤ 0.81 in any single one), after $\sigma \times k \times \mathcal{L}$ scaling. **pyhf 95% CL limit:** $\mu_{\text{obs}} = \mu_{\text{exp}} = 2.0$ (the muscan ceiling, so the true limit is > 2), with $\text{CL}_s = 0.74$ — i.e. the point (200, 150) is **not excluded**, and not marginally so.

Chapter 6

Sensibility and sanity analysis

6.1 What a sane result looks like

We check each stage’s output for physical plausibility before trusting the final number:

- **Cross-section:** the ISR-tagged slepton LO σ should be a few $\times$ 10 fb (the inclusive NLO+NLL slepton σ at $m = 200$ is ~ 60 fb; the 1-jet-tagged LO subset being smaller is consistent). Observed: 20.6 fb (1k), 20.5 fb (50k) — stable.
- **Acceptance:** a few percent is plausible for a compressed soft-lepton selection.
- **Yields:** should populate the signal regions smoothly at 50k (unlike the sparse 1k).
- **Limit:** μ should be finite (resolved within the scan), not pinned at the ceiling.

6.2 Comparison to the published reference

Important: the comparison must be against the *same* analysis we ran (`EwkCompressed2018`, the 2020 ATLAS compressed-SUSY search the paper reinterprets) — not against the different 2024 BDT-based slepton analysis whose files also happen to be in the workspace. For the point (200, 150), $\Delta m = 50$ GeV sits at the *upper edge* of the compressed (soft-lepton) search’s sensitivity, where the ATLAS slepton-bino exclusion for $m(\tilde{\ell}) \approx 200$ GeV turns over. Our pipeline’s verdict — **not excluded**, with only ~ 7 signal events spread across the discriminating regions — is therefore the physically expected outcome: the point lies at or just beyond the exclusion boundary.

6.3 Did we reproduce the paper?

The pipeline is faithfully reproduced. Every stage ran in the paper’s actual containers (MadGraph 2.9.3, the ATLAS Delphes card, ATLAS SimpleAnalysis `EwkCompressed2018`, the bundled `pyhf` Slepton likelihood) under `mapyde`, and produced self-consistent, physically reasonable intermediate results (a stable cross-section, a plausible acceptance, a valid statistical limit). What we did *not* attempt is a bit-for-bit match of the published exclusion contour at this single point, which would require: (i) the ATLAS 2-jet CKKW-L-merged sample rather than `mapyde`’s 1-jet `isrslep`; (ii) NLO+NLL rather than $\text{LO} \times k$; (iii) the paper’s hand-tuned low- p_T lepton efficiencies in Delphes; and (iv) a finer muscan and the `EwkCompressed2018` contour (not the unrelated 2024 BDT-analysis files in the workspace) for the comparison. Within those documented caveats, the reproduction is sound.

Chapter 7

Generalising the pipeline: Rivet, plots, and Contur

The slepton study above is tied to one ATLAS analysis (a SimpleAnalysis selection + a published pyhf likelihood). To make the pipeline work for an *arbitrary* analysis, the analysis stage is replaced by **Rivet**. A Rivet routine is a self-contained encoding of a published analysis: it reads the showered events (HepMC), applies the analysis’s object definitions, selection, and — for search routines — its own detector smearing, and fills the analysis’s observables, which ship with the measured *reference data*. The chain becomes analysis-agnostic:

MadGraph $\rightarrow$ Pythia8 $\rightarrow$ Rivet routine $\rightarrow$ `rivet-mkhtml` $\rightarrow$ Contur.

There are ~ 2000 routines bundled with Rivet, so the only per-analysis input is the routine ID (plus the model cards). Two buckets of the original pipeline map cleanly onto Rivet: the *analysis* bucket is the routine, and the *reusability/visualization* bucket is `rivet-mkhtml`, which overlays the model prediction on the data.

7.1 Why a plot is the deliverable

A scientist does not read signal-region yields; they look at a distribution and ask “is the signal there?”. `rivet-mkhtml` answers this directly: it draws each observable with the model prediction over the measured data and a ratio panel. Where the model rises to meet or exceed the data in a discriminating region, the model would have produced a visible excess — it is constrained or excluded; where it sits far below, the analysis is blind to it.

7.2 Worked demonstration: a gluino vs an ATLAS jets+MET search

As an arbitrary test, gluino-pair production ($\tilde{g} \rightarrow q\bar{q}\tilde{\chi}_1^0$, $m_{\tilde{g}} = 1000$, $m_{\tilde{\chi}_1^0} = 100$ GeV) was generated with MadGraph ($\sigma_{\text{LO}} = 0.20$ pb), showered to HepMC3, and passed to the Rivet routine `ATLAS_2016_I1458270` (the 0-lepton jets+ E_T^{miss} search, 13 TeV, 3.2 fb^{-1}). `rivet-mkhtml` produced the effective-mass distributions per signal region: in the tight, high-jet-multiplicity regions the predicted signal sits an order of magnitude *above* the observed data at high m_{eff} . Read off the plot, this gluino point is clearly excluded — consistent with the published search reaching $m_{\tilde{g}} \approx 1.35$ TeV for a light LSP.

7.3 The exclusion stage: Contur, and its scope

Contur turns the comparison into a number. It is built for *measurements*: it knows the Standard-Model prediction and the data for a large library of measured distributions, and reports the confidence with which the model’s extra contribution is excluded. For a measurement routine, **contur** on the YODA returns a CLs directly. For a *search* routine (like the demo), Contur runs but adds no likelihood unless that search is in its database — the formal limit then comes from **pyhf** (next chapter), built either from the search’s published likelihood or from a counting model on its bundled background and data. Contur is the route for *measurement* routines; for a search, **pyhf** is. This is a scope property to be aware of when choosing the exclusion method.

7.4 Two co-equal analysis routines: Rivet and SimpleAnalysis

The analysis stage offers two first-class routine types, chosen by what the target analysis publishes. **Rivet** runs entirely from native conda tools (fast, no emulation), covers ATLAS *and* CMS, and ships the reference data so the comparison plots come for free. **SimpleAnalysis** is the signal-region–yield framework that many ATLAS/CMS SUSY analyses publish a routine for; its per-region yields map directly onto a published **pyhf** likelihood for the strongest possible limit, and it runs in the ATLAS container (hence the podman/emulation machinery of the earlier chapters). Step 4 of the workflow picks whichever the analysis provides, and both feed the same statistical stage below.

Chapter 8

The statistical exclusion: pyhf, the data, and the limit

The plot shows whether a signal *looks* excluded; a number makes it quantitative. The pipeline’s statistical stage uses **pyhf** — the standard tool — and never a hand-rolled model. The output is the 95% confidence-level upper limit on the signal strength μ (the scale factor on the nominal signal cross-section): the model is *excluded* when the observed $\mu^{95} < 1$.

8.1 Where the numbers come from

A limit needs the analysis’s own observed counts and Standard-Model background per signal region, and — when published — a serialized likelihood. The pipeline gets them bundled-first:

- **Rivet’s bundled reference data** already carries, for a search routine, the observed data (y01) and the SM background with its uncertainty (y02). For the core case no download is needed: `rivet_ref_yields.py` integrates these above each signal region’s threshold to give (observed, $\text{background} \pm \sigma$), paired with the signal yield from the routine’s own region counter.
- **HEPData** supplies a serialized likelihood when one exists. Its human download pages sit behind a bot-challenge, but the *JSON API* (`?format=json`) is open, so `hepdata_fetch.py` discovers a record’s tables and flags any **pyhf** likelihood; the rare archive download falls to a browser. Many pre-2018 searches publish no likelihood — then the counting model is the route.

8.2 Two model modes

Serialized likelihood. When the analysis publishes a **pyhf** workspace (as for the slepton study), the signal patch from `sa2json` is applied to the background-only workspace and fitted directly — the analysis’s full statistical model, with all its nuisance parameters (here 38 channels, 191 parameters).

Counting model. When no likelihood is published (as for the 2016 gluino search), a simple single-bin counting model is built per signal region from (observed, $\text{background} \pm \sigma$, signal), and the limit is quoted from the single *most sensitive* region (best expected CL_s) — the standard prescription when regions overlap and cannot be combined.

Both were exercised on the worked examples. The gluino ($m = 1$ TeV) counting model gives an observed $\mu^{95} = 0.10$ (most-sensitive region: the 5-jet SR) — *excluded* roughly tenfold, consistent with the published reach of ~ 1.3 – 1.5 TeV. The slepton serialized likelihood gives $\mu^{95} = 6.4$ — *not excluded*, as expected for a compressed point ($\Delta m = 50$) beyond the search’s sensitivity.

8.3 The $\mu = 2$ lesson: a fixed grid is not a limit

The slepton’s earlier exclusion curve stopped at $\mu = 2$ and was easy to misread as “the limit is 2”. It is not. `mapyde`’s `muscan` evaluates CL_s on a *fixed* grid $\mu \in [0.1, 2.0]$; at the grid’s ceiling CL_s was still ≈ 0.52 , far above the 0.05 that defines the 95% CL crossing. The scan simply ended before the limit — the model is weakly constrained and the true limit lies well past the grid. `pyhf_exclude.py` removes the ceiling: it brackets μ by doubling until $CL_s < 0.05$, then interpolates the crossing, and finds $\mu^{95} = 6.4$. The lesson for an agent: a limit pinned to the edge of a fixed scan, with CL_s still large there, is an artefact of the grid, not a result.

8.4 Plots a scientist can read

Two properties make the plots usable. First, they are *comparable to the experiment’s own*: `rivet-mkhtml` overlays the model on the measured data with a ratio panel, the same view the collaboration publishes. Second, they are *named*: `rivet-mkhtml` keys files by the routine’s internal ids (`d04-x01-y01`) or terse region tags (`2j1`, `CF-2j1`), which a reader cannot parse, so `name_plots.py` writes a copy under a standard scheme, `<routine>__<origID>__<label>` (e.g. `...d08-x01-y01_meff-incl.SR-5j`), keeping the original id for traceability and adding an `INDEX.md` legend. The naming convention is a fixed, embedded step, not a per-run improvisation.

8.5 Rigour without a reference

The runs in this guide test models the analyses’ authors also studied, so each limit could be checked against a published reach — and that is exactly what makes them useful here: they are *scaffolding* that confirms the chain behaves. But the workflow’s purpose is to test models nobody has published a result for, where there is nothing to compare against. Rigour then has to be *intrinsic*: the statistical model is the analysis’s own (its likelihood, or its observed + background), paired with the matching selection; the signal cross-section carries its NLO+NLL normalisation; the limit reaches the true $CL_s = 0.05$ crossing; and the signal acceptance comes from a chain that was *certified once* against a published cutflow, so later new-model runs inherit that trust. The agent’s protocol for this — per-run intrinsic checks plus a one-time certification — is `../workflow/checklists/validation.md`. The known-result comparisons below and above are method-validation, not the deliverable.

Chapter 9

The agentic trial: dogfooding the workflow

The point of the project is that an *agent* drives the workflow. To test that, a fresh subagent was given the documentation and the staged model inputs and asked to execute steps 3–7 on a new case, reporting every place the instructions were unclear, wrong, or insufficient — while Opus did the judgement steps and logged the stumbles.

9.1 The case, and why

The clean automated exclusion needs a routine that bundles *aligned* observed and background per region with a simple region $\leftrightarrow$ table map. Scouting Rivet’s bundled-data ATLAS/CMS searches showed this is not universal: a CMS compressed-electroweakino search bundles only a cutflow (NOHEPDATA); an ATLAS electroweak recursive-jigsaw search is an 800-line routine with no clean region $\rightarrow$ table map; a 7 TeV high-jet search bundles three identical MC copies and mismatched data/ background binnings. So the trial reused the clean ATLAS_2016_I1458270 routine but tested a *different model the same paper interprets* — **squark-pair** ($\tilde{q} \rightarrow q\tilde{\chi}_1^0$, $m_{\tilde{q}} = 800$, eight degenerate light flavours, gluino decoupled). Squarks give two jets+ E_T^{miss} , not the gluino’s four-to-six, so the test also checks that the pipeline finds the right signal regions for a different topology.

9.2 What the agent produced

The subagent generated $\sigma_{\text{LO}} = 0.30$ pb, showered, ran the routine, made and named the 21 plots, built the counting inputs from the bundled reference, and ran **pyhf**: observed $\mu^{95} = 0.28$, most-sensitive region **2jl, excluded**. The signal landed in the *low*-multiplicity 2-jet regions — the opposite of the gluino demo’s 5-/6-jet regions — so the chain tracked the model-dependent sensitivity correctly. Here a chosen benchmark happens to coincide with a model the authors also studied (squarks excluded to ≈ 1.0 – 1.05 TeV), so the “excluded” verdict could be checked against that — a useful confirmation that the chain behaves on a known case. That cross-check is scaffolding, not the deliverable: the workflow exists to test models the authors never considered, where no such comparison exists and the limit must stand on its inputs (Chapter on intrinsic rigour / `validation.md`).

9.3 Where the instructions stumbled (and the fixes)

The docs were followable end-to-end; three small gaps were found and fixed:

1. **Run-card settings under-specified.** Step 3 showed the `sed` for `nevents` but left `ebeam/iseed/use_syst` as a bare comment. Fix: show all four `sed` lines explicitly, with the rule that the beam energy is half the routine's $\sqrt{s}$.
2. **A relative output path after a `cd`.** Step 5's `rivet-mkhtml -o <rundir>/plots` ran after `cd <rundir>/build`, so the plots would land in the wrong place. Fix: drop the `cd` and pass full paths.
3. **Bare python for the helper scripts.** There is no `python` on the base `PATH`; the helpers must run as `$CONDA run -n rivet python ...` like every other command. Fix applied to the `visualize` and `acquire-data` steps and their checklists.

That a fresh agent completed a new reinterpretation with only three minor, mechanical corrections is the result that matters: the workflow is genuinely agent-drivable, and dogfooding is how the remaining rough edges get found. The full log is `../trial-runs/2026-06-08_ATLAS_2016_I1458270_squar`

Chapter 10

Observations & lessons for agentic-workflow design

1. The faithful pipeline *is* runnable on a constrained laptop, but only after the five infrastructure fixes of Chapter 4; an agent must own these explicitly.
2. Emulation makes it correct but slow — the agent should always smoke-test small, then scale.
3. `mappyde run all` silently omits the `simpleanalysis/sa2json` stages, so the agent must run the six stages explicitly to reach a limit.
4. Statistics matter: a smoke test cannot resolve a limit; the agent must recognise the “ μ at scan ceiling” signature and scale up.
5. Asset hygiene: the workspace contained files from a *different* analysis; the agent must pair the SimpleAnalysis selection with the matching likelihood and reference.
6. Commands must be *literally* runnable: the dogfood agent tripped only on an interpreter prefix (`$CONDA run -n rivet python`), a relative `-o` path after a `cd`, and an under-specified `sed` — mechanical, not conceptual. Agent-drivability is a property of the exact command text, not just the prose around it.
7. Routines are not interchangeable: the clean automated exclusion needs a routine that bundles *aligned* observed+background per region with a simple region $\leftrightarrow$ table map. An agent should check this at routine-selection time, not discover it at the exclusion step.

Chapter 11

Limitations & next steps

- LO matrix element + k -factor; emulation-limited throughput for the containerized path.
- The 1-jet `isrslep` sample differs from the ATLAS 2-jet merged sample (a known, documented choice of `mapyde`).
- The counting-model exclusion sums the bundled per-bin background uncertainties (with a relative floor) rather than using a published inclusive-region uncertainty; the serialized likelihood is exact where one is published.
- The clean automated exclusion needs a routine that bundles *aligned* observed+background per region with a simple region $\leftrightarrow$ table map; routines that do not (some CMS searches bundle only a cutflow; some are 800-line jigsaw routines) need per-routine work.
- Next: scan additional points and mass planes, automate the publication comparison, and widen the agentic driver beyond the cases tried here.

Chapter 12

Validation, audit, and readiness

A reinterpretation tool is only useful if it is *faithful*, and — since it tests models nobody published a result for — that faithfulness must be established once, intrinsically, not inferred per run. The bar is set by the LHC Reinterpretation Forum (arXiv:2003.07868): reproduce a published acceptance $\times$ efficiency, use the analysis’s own statistical model and complete data, handle complex analyses, record provenance, and be auditable.

12.1 Acceptance validation against published cutflows

Acceptance $\times$ efficiency is a ratio (selected/generated), independent of cross-section and luminosity, so it isolates the generation $\rightarrow$ shower $\rightarrow$ detector $\rightarrow$ selection chain. Comparing our $A\times\epsilon$ to ATLAS’s published grids (fetched with `hepdata-cli`) gives the gluino (1 TeV) **7/7 signal regions within $\pm 30\%$** — SR2jl agrees to **0.3%** (0.1543 vs 0.1547). The squark’s sensitive 2-jet regions agree to 2%; its high-multiplicity regions run $\sim 35\%$ low, the expected fingerprint of not merging extra jets. The compressed slepton’s acceptance is within $\sim 2\times$ of ATLAS, the residual traced to the 1-jet ISR sample — so the weak slepton limit is a quantified modelling gap, not a broken chain.

12.2 An independent cross-check

SModelS (an independent recasting code, with digitised ATLAS+CMS efficiency maps) run on the same gluino reports it excluded, with $r_{\text{obs}} = 8.07$ for the very analysis we used; our pyhf gives $\mu^{95} = 0.10$, i.e. $1/\mu^{95} \approx 10$ — the two agree to $\sim 20\%$, the expected level for independent recasts.

12.3 A self-auditing readiness score

`framework/audit.py` scores ten measurable dimensions (validation, statistical rigour, complete data, complexity, provenance, visual fidelity, cross-check, documentation, dependencies, known limitations) into `AUDIT.md`. It is evidence-based — it reads the actual certifications, runs, and provenance stamps — so the number moves only when the pipeline genuinely improves. The current score is **96%**; the open items (NLO cross-sections, the MadAnalysis5/CheckMATE builds, a fresh recursive-jigsaw run) are tracked in `KNOWN-LIMITATIONS.md`, not hidden.

Appendix A

Glossary

LHE / HepMC / ROOT

the parton-level / particle-level / detector-level event file formats.

cross-section σ

interaction rate; expected events = $\sigma \times$ luminosity.

signal region

a kinematic selection where signal is expected to concentrate.

signal strength μ

scale factor on the signal; the 95% CL upper limit on μ is the result.

CKKW-L

a scheme to merge matrix elements of different jet multiplicity with the shower.

container / image

a packaged, reproducible software environment run by podman/docker.

emulation

running amd64 binaries on arm64 via a translation layer (slower).

Appendix B

Reproduce these runs

See `../workflow/SESSION-MANUAL.md` for the self-contained procedure and the agent prompt, and `../trial-runs/` for the recorded runs (each with its `RESULT.md`).

Appendix C

References

1. Stark, Aristimuno Ots, Hance, *Reduce, Reuse, Reinterpret*, arXiv:2306.11055.
2. Alwall *et al.*, MadGraph5_aMC@NLO, JHEP 07 (2014) 079.
3. ATLAS, *Compressed electroweak SUSY searches at 13 TeV*, Phys. Rev. D 101 (2020) 052005.
4. The mapyde package and its bundled cards/likelihoods.